Arquitetura e Plano de Implementação: Backend Spring Boot & Serviço de Relatórios Python

Este documento descreve a arquitetura técnica, divisão de responsabilidades, estratégia de integração e o plano detalhado de implementação para o backend do **SmartFlow IA (OTIS)**, combinando **Java Spring Boot** (para regras de negócio, dados e segurança) com **Python** (para geração de dashboards, relatórios analíticos, PDFs e planilhas formatadas).

1. Visão Geral da Arquitetura

O sistema adota uma **arquitetura em camadas orientada a serviços e microsserviços**, onde o frontend se conecta unicamente à API Spring Boot para operações e downloads, e o Spring Boot orquestra a geração de documentos pesados delegando ao motor analítico Python:

FRONTEND (SPA) **React + Vite + TypeScript** Portas 3000 / 5173	BACKEND CORE (JAVA) **Spring Boot 3.3.4 (REST API & Auth)** Porta 8080 • Spring Data JPA	ENGINE DE RELATÓRIOS (PYTHON) **FastAPI • ReportLab • OpenPyXL** Porta 8000 • Pandas
• Dashboards Executivos • Chamados & Operação • Botões de Exportação	• Regras de Negócio OTIS • Orquestrador RestClient • Banco PostgreSQL / H2	• PDFs Executivos de Alta Resolução • Planilhas Excel com DRE e Fórmulas • Gráficos & Estatísticas

Divisão de Responsabilidades por Componente:

Componente	Tecnologia	Papel Principal
Backend Core	Java 17 Spring Boot 3.3.x	APIs REST, regras de negócio do SmartFlow, gestão de chamados, técnicos, equipamentos, telemetria, autenticação/autorização (JWT), persistência de dados.
Reports & Analytics Service	Python 3.12 FastAPI	Processamento analítico, agregação estatística, formatação de planilhas Excel corporativas (.xlsx com estilos Otis), geração de relatórios executivos em PDF com gráficos visuais e exportações de dashboards.
Banco de Dados	PostgreSQL (Prod) H2 (Dev)	Armazenamento de dados relacionais e transacionais com migrações ou mapeamento JPA.
Frontend	React + Vite + TypeScript	Interface visual que consome as APIs do Spring Boot e efetua download direto dos relatórios.

2. Decisões Técnicas e Pontos de Revisão

IMPORTANTE — Compatibilidade de Versões do Java & Spring Boot:

O ambiente local possui **Java 17 (OpenJDK Temurin-17)** e **Python 3.12.10**. O pom.xml original continha versão 4.1.1 e Java 21 inválidos, que foram ajustados para **Spring Boot 3.3.4** e **Java 17**, garantindo compilação nativa com o Maven Wrapper (mvnw.cmd).

NOTA — Padrão de Comunicação Spring Boot ↔ Python:

A arquitetura adota uma API interna em **FastAPI** rodando em background na porta 8000. O Spring Boot valida a sessão do usuário, aplica permissões hierárquicas (RBAC: Presidente, Gerente, Supervisor), extrai o escopo de dados e chama o microserviço Python via RestClient. O arquivo gerado é repassado ao cliente via HTTP streaming com cabeçalho Content-Disposition: attachment.

3. Estrutura Proposta de Pastas

```
otis-smartflow/ ■■■ backend/ ■ ■■■ smartflow-api/ # Backend Java Spring Boot ■ ■ ■■■ pom.xml # Spring Boot 3.3.4, Java 17, JPA, Web, Security, Lombok ■ ■ ■■■ src/ ■ ■ ■■■ main/ ■ ■ ■ ■■■ java/br/com/otis/smartflow/ ■ ■ ■ ■ ■■■ SmartflowApiApplication.java ■ ■ ■ ■ ■■■ config/ # CorsConfig, SecurityConfig, RestClientConfig ■ ■ ■ ■ ■■■ controller/ # Equipments, Calls, Technicians, ReportsController ■ ■ ■ ■ ■■■ dto/ # Request e Response DTOs ■ ■ ■ ■ ■■■ model/ # Entidades JPA (Equipment, Call, Technician, etc.) ■ ■ ■ ■ ■■■ repository/ # Spring Data JPA Repositories ■ ■ ■ ■ ■■■ service/ # Regras de negócio e integração com Python ■ ■ ■ ■ ■■■ client/ # PythonReportsClient.java ■ ■ ■ ■■■ resources/ ■ ■ ■ ■■■ application.yml ■ ■ ■ ■■■ smartflow-reports/ # Serviço de Relatórios Python ■ ■■■ requirements.txt # fastapi, uvicorn, pandas, openpyxl, reportlab, matplotlib ■ ■■■ main.py # Aplicação FastAPI ■ ■■■ config.py # Configurações e tokens internos ■ ■■■ routers/ ■ ■ ■■■ reports.py # Endpoints de geração de relatórios ■ ■■■ generators/ ■ ■ ■■■ executive_pdf.py # Relatório Executivo Nacional (PDF) ■ ■ ■■■ sla_excel.py # Cumprimento de SLA por Polo (Excel/CSV) ■ ■ ■■■ financial_dossier.py # Dossiê Financeiro & DRE (Excel) ■ ■ ■■■ predictive_report.py # Inventário Preditivo & Falhas (Excel/PDF) ■ ■ ■■■ charts.py # Gráficos embutidos (Matplotlib) ■ ■■■ templates/ # Estilos, cabeçalhos, logos da OTIS ■ ■■■ frontend/ ■■■ prototipo/ # Frontend React já existente
```

4. Modelagem de Relatórios & Dashboards Suportados

Baseado nos relatórios definidos no protótipo (ReportsModule.tsx), o serviço Python fornecerá:

1. Relatório Executivo Nacional (Presidência)

- **Formato:** PDF diagramado em alta resolução.
- **Conteúdo:** Sumário executivo, disponibilidade global da frota (%), taxa de SLA, gráficos comparativos de custos e métricas preditivas.

2. Relatório de Cumprimento de SLA por Polo

- **Formato:** Excel estilizado (.xlsx) e CSV.
- **Conteúdo:** Desempenho por supervisor/região, tempo de deslocamento (TA), tempo de solução (TB), chamados fora do prazo e índice de reincidência.

3. Dossiê Financeiro & DRE de Contratos

- **Formato:** Excel com abas e fórmulas (.xlsx).
- **Conteúdo:** Receita por contrato, consumo de peças, horas técnicas aplicadas, margem operacional de cada contrato e alertas de risco financeiro.

4. Inventário Preditivo & Padrões de Falha

- **Formato:** PDF / Excel.
- **Conteúdo:** Ranking de equipamentos por score de risco (0-100), componentes sob fadiga (portas AT120, drives regenerativos, cabos) e recomendações da IA.

5. Auditoria de Decisões: Supervisor vs IA

- **Formato:** PDF com tabela de evidências.
- **Conteúdo:** Histórico de desvios operacionais entre a indicação do SmartFlow IA e a alocação manual feita pelo supervisor, com justificativas.

5. Plano de Implementação Passo a Passo

Fase 1: Configuração e Correção do Spring Boot (Java)

- [✓] Atualizar pom.xml: Definir spring-boot-starter-parent 3.3.4 e java.version 17.
- [✓] Adicionar dependências essenciais: spring-boot-starter-web, spring-boot-starter-data-jpa, h2 (dev), postgresql (prod), lombok, springdoc-openapi-starter-webmvc-ui (Swagger).
- [✓] Configurar application.properties com profile para banco em memória H2 e suporte a PostgreSQL.
- [✓] Compilar e validar a execução do Spring Boot com .\mvnw.cmd clean test.

Fase 2: Implementação de Domínio e REST API no Spring Boot

- [✓] Criar entidades de domínio: Equipment, Technician, CallTicket, Contract, ReportRequest.
- [✓] Criar repositórios JPA e serviços de consulta para alimentar as extrações de dados.
- [✓] Configurar RestClient para comunicação HTTP com o serviço Python.
- [✓] Criar ReportsController: GET /api/v1/reports/types e GET /api/v1/reports/download/{reportType}.

Fase 3: Criação do Microsserviço de Relatórios em Python

- [✓] Criar diretório Backend/smartflow-reports com requirements.txt (fastapi, uvicorn, pandas, openpyxl, reportlab, matplotlib).
- [✓] Implementar geradores de relatórios: Módulo Excel (openpyxl) com paleta OTIS (tons de azul, ardósia, formatação de moeda).
- [✓] Implementar gerador de relatórios executivos em PDF (reportlab) com diagramação limpa, tabelas zebradas e paginação.
- [✓] Criar endpoints FastAPI correspondentes e testar geração autônoma.

Fase 4: Integração Frontend ↔ Spring Boot ↔ Python

- [✓] Ajustar ReportsModule.tsx no Frontend para disparar download real na API Spring Boot.
- [✓] Adicionar feedback de progresso (loading spinner/toast de progresso) durante o download.

6. Plano de Verificação & Comandos de Execução

```
# Testes Automatizados do Spring Boot (Java) cd Backend/smartflow-api .\mvnw.cmd test # Iniciar Serviço de Relatórios Python (Porta 8000) cd backend/smartflow-reports uvicorn main:app --port 8000 --reload # Documentação Swagger Python: http://localhost:8000/docs # Iniciar Backend Java Spring Boot (Porta 8080) cd backend/smartflow-api .\mvnw.cmd spring-boot:run # Swagger UI Spring Boot: http://localhost:8080/swagger-ui.html # Console H2 Database: http://localhost:8080/h2-console # Iniciar Frontend React + Vite (Porta 3000 / 5173) cd frontend/prototipo npm run dev
```

Verificação Manual no Navegador:

1. Acesse o Swagger UI (http://localhost:8080/swagger-ui.html) e execute o endpoint /api/v1/reports/download/executivo-nacional.
2. No Frontend, acerte o módulo de Relatórios e clique em **Baixar Relatório** para testar o download do arquivo PDF/Excel gerado em tempo real.